LAB ASSIGNMENT 5.1 & 6

M.RANADEEP

2303A51683

Batch-24

Task 1:

```
class Employee:
    def __init__(self, empid, empname, designation, basic_salary, exp):
        self.empid = empid
        self.empname = empname
        self.designation = designation
        self.basic_salary = basic_salary
        self.exp = exp

    def display_details(self):
        print(f"Employee ID: {self.empid}")
        print(f"Employee Name: {self.empname}")
        print(f"Designation: {self.designation}")
        print(f"Basic Salary: {self.basic_salary}")
        print(f"Experience: {self.exp} years")

    def calculate_allowance(self):
        if self.exp > 10:
            allowance = 0.20 * self.basic_salary
        elif 5 <= self.exp <= 10:
            allowance = 0.10 * self.basic_salary
        else:
            allowance = 0.05 * self.basic_salary
        return allowance
# Example usage:
```

```
empobj1  = Employee(empid=101, empname="John Doe", designation="Manager",
basic_salary=50000, exp=8)

empobj1.display_details()

allowance = empobj1.calculate_allowance()

print(f"Calculated Allowance: {allowance}")
```

Output:

Employee ID: 101

Employee Name: John Doe

Designation: Manager

Basic Salary: 50000

Experience: 8 years

Calculated Allowance: 5000.0

Task2 :

```
class ElectricityBill:
    def __init__(self, customer_id, name, units_consumed):
        self.customer_id = customer_id
        self.name = name
        self.units_consumed = units_consumed

    def display_details(self):
        print(f"Customer ID: {self.customer_id}")
        print(f"Name: {self.name}")
        print(f"Units Consumed: {self.units_consumed}")

    def calculate_bill(self):
        if self.units_consumed <= 100:
            bill_amount = self.units_consumed * 5
        elif 101 <= self.units_consumed <= 300:
            bill_amount = self.units_consumed * 7
```

```
        else:
            bill_amount = self.units_consumed * 10
        return bill_amount
# Example usage:
billobj1 = ElectricityBill(customer_id=201, name="Alice Smith", units_consumed=150)
billobj1.display_details()
total_bill = billobj1.calculate_bill()
print(f"Total Bill Amount: ₹{total_bill}")
```

Output:

Customer ID: 201

Name: Alice Smith

Units Consumed: 150

Total Bill Amount: ₹1050

Task 3:

```
class Product:
    def __init__(self, product_id, product_name, price, category):
        self.product_id = product_id
        self.product_name = product_name
        self.price = price
        self.category = category

    def display_details(self):
        print(f"Product ID: {self.product_id}")
        print(f"Product Name: {self.product_name}")
        print(f"Price: ₹{self.price}")
        print(f"Category: {self.category}")

    def calculate_discount(self):
        if self.category == "Electronics":
```

```
            discount = 0.10 * self.price
        elif self.category == "Clothing":
            discount = 0.15 * self.price
        elif self.category == "Grocery":
            discount = 0.05 * self.price
        else:
            discount = 0
        return discount

# Example usage:
product1 = Product(product_id=301, product_name="Laptop", price=50000, category="Electronics")
product1.display_details()
discount = product1.calculate_discount()
print(f"Discount Amount: ₹{discount}")
print(f"Final Price After Discount: ₹{product1.price - discount}")
```

Output:

Product ID: 301

Product Name: Laptop

Price: ₹50000

Category: Electronics

Discount Amount: ₹5000.0

Final Price After Discount: ₹45000.0

Task 4:

```
class LibraryBook:
    def __init__(self, book_id, title, author, borrower, days_late):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.borrower = borrower
```

```
        self.days_late = days_late

    def display_details(self):
        print(f"Book ID: {self.book_id}")
        print(f"Title: {self.title}")
        print(f"Author: {self.author}")
        print(f"Borrower: {self.borrower}")
        print(f"Days Late: {self.days_late}")

    def calculate_late_fee(self):
        if self.days_late <= 5:
            late_fee = self.days_late * 5
        elif 6 <= self.days_late <= 10:
            late_fee = self.days_late * 7
        else:
            late_fee = self.days_late * 10
        return late_fee
# Example usage:
book1 = LibraryBook(book_id=401, title="Python Programming", author="Jane Doe", borrower="Bob Johnson", days_late=8)
book1.display_details()
late_fee = book1.calculate_late_fee()
print(f"Late Fee: ₹{late_fee}")
```

Output:

Book ID: 401

Title: Python Programming

Author: Jane Doe

Borrower: Bob Johnson

Days Late: 8

Late Fee: ₹56

Task 5:

```
def student_report(student_data):
    report = []
    for student, marks in student_data.items():
        average_score = sum(marks) / len(marks)
        status = "Pass" if average_score >= 40 else "Fail"
        report.append({
            "Student": student,
            "Average Score": average_score,
            "Status": status
        })
    return report
# Example usage:
students = {
    "Alice": [45, 78, 89],
    "Bob": [34, 23, 40],
    "Charlie": [90, 88, 92]
}
report = student_report(students)
for student_summary in report:
    print(student_summary)
```

Output:

```
{'Student': 'Alice', 'Average Score': 70.66666666666667, 'Status': 'Pass'}
{'Student': 'Bob', 'Average Score': 32.333333333333336, 'Status': 'Fail'}
{'Student': 'Charlie', 'Average Score': 90.0, 'Status': 'Pass'}
```

Task 6:

```
class TaxiRide:
    def __init__(self, ride_id, driver_name, distance_km, waiting_time_min):
```

```
        self.ride_id = ride_id
        self.driver_name = driver_name
        self.distance_km = distance_km
        self.waiting_time_min = waiting_time_min

    def display_details(self):
        print(f"Ride ID: {self.ride_id}")
        print(f"Driver Name: {self.driver_name}")
        print(f"Distance (km): {self.distance_km}")
        print(f"Waiting Time (min): {self.waiting_time_min}")

    def calculate_fare(self):
        if self.distance_km <= 10:
            fare = self.distance_km * 15
        elif 10 < self.distance_km <= 30:
            fare = (10 * 15) + ((self.distance_km - 10) * 12)
        else:
            fare = (10 * 15) + (20 * 12) + ((self.distance_km - 30) * 10)

        waiting_charge = self.waiting_time_min * 2
        total_fare = fare + waiting_charge
        return total_fare
# Example usage:
ride1 = TaxiRide(ride_id=501, driver_name="David Lee", distance_km=25, waiting_time_min=5)
ride1.display_details()
total_fare = ride1.calculate_fare()
print(f"Total Fare: ₹{total_fare}")
```

Output:

Ride ID: 501

Driver Name: David Lee

Distance (km): 25

Waiting Time (min): 5

Total Fare: ₹340

Task 7:

```
def statistics_subject(scores_list):
    if not scores_list:
        return {
            "Highest Score": 0,
            "Lowest Score": 0,
            "Average Score": 0,
            "Passed Students": 0,
            "Failed Students": 0
        }

    highest_score = max(scores_list)
    lowest_score = min(scores_list)
    average_score = sum(scores_list) / len(scores_list)
    passed_students = sum(1 for score in scores_list if score >= 40)
    failed_students = len(scores_list) - passed_students

    return {
        "Highest Score": highest_score,
        "Lowest Score": lowest_score,
        "Average Score": average_score,
        "Passed Students": passed_students,
        "Failed Students": failed_students
    }
# Example usage:
scores = [45, 78, 89, 34, 23, 40, 90, 88, 92, 55, 60, 30, 20, 15, 80, 85, 95, 100, 35, 25]
statistics = statistics_subject(scores)
```

```
for key, value in statistics.items():
    print(f"{key}: {value}")
```

Output:

Highest Score: 100

Lowest Score: 15

Average Score: 58.95

Passed Students: 13

Failed Students: 7

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task 8:

```
def is_prime_optimized(num):
    """
    This function checks if a number is prime using an optimized approach.

    Parameters:
    num (int): The number to check for primality.

    Returns:
    bool: True if num is prime, False otherwise.
    """
    if num <= 1:
        return False
    if num <= 3:
        return True
    if num % 2 == 0 or num % 3 == 0:
        return False
    i = 5
    while i * i <= num:
        if num % i == 0 or num % (i + 2) == 0:
```

```
            return False
        i += 6
    return True

# Naive Approach to Check Prime Number
def is_prime_naive(num):
    """
    This function checks if a number is prime using a naive approach.

    Parameters:
    num (int): The number to check for primality.

    Returns:
    bool: True if num is prime, False otherwise.
    """
    if num <= 1:
        return False
    for i in range(2, num):
        if num % i == 0:
            return False
    return True

# Example usage:
number = 29
print(f"Naive approach: Is {number} prime? {is_prime_naive(number)}")
print(f"Optimized approach: Is {number} prime? {is_prime_optimized(number)}")
```

Output:

Naive approach: Is 29 prime? True

Optimized approach: Is 29 prime? True

Task 9:

```
def fibonacci(n):
    """
    This function returns the nth Fibonacci number using recursion.

    The Fibonacci sequence is defined as follows:
    F(0) = 0
    F(1) = 1
    F(n) = F(n-1) + F(n-2) for n > 1

    Parameters:
    n (int): The position in the Fibonacci sequence to retrieve.

    Returns:
    int: The nth Fibonacci number.
    """

    # Base case: return 0 for the 0th Fibonacci number
    if n == 0:
        return 0
    # Base case: return 1 for the 1st Fibonacci number
    elif n == 1:
        return 1
    # Recursive case: return the sum of the two preceding Fibonacci numbers
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
# Example usage:
n = 10
print(f"The {n}th Fibonacci number is: {fibonacci(n)}")
```

Output:

The 10th Fibonacci number is: 55

Task 10:

```
def read_and_process_file(file_path):
    """
    This function reads a file and processes its data.

    Parameters:
    file_path (str): The path to the file to be read.

    Returns:
    list: A list of processed data lines.
    """
    processed_data = []
    try:
        # Attempt to open the file
        with open(file_path, 'r') as file:
            for line in file:
                # Process each line (for example, stripping whitespace)
                processed_line = line.strip()
                processed_data.append(processed_line)
    except FileNotFoundError:
        # Handle the case where the file does not exist
        print(f"Error: The file at '{file_path}' was not found.")
    except PermissionError:
        # Handle the case where there are permission issues
        print(f"Error: Permission denied when trying to read the file at '{file_path}'.")
    except Exception as e:
        # Handle any other exceptions that may occur
        print(f"An unexpected error occurred: {e}")
```

```
    return processed_data
# Example usage:
file_path = 'sample.txt'
data = read_and_process_file(file_path)
print("Processed Data:", data)
```

Output:

Processed data[ ]